A Rust BBHash-Style Minimal Perfect Hash Function for Static Lookup Workloads

Devanshu Singhvi
UTORid: singhv67

April 15, 2026

Abstract. A minimal perfect hash function (MPHF) maps a fixed set of n keys bijectively to $[0, n)$ with no collisions. MPHFs are useful in static systems such as dictionary encoding, read-only indexes, and graph pipelines because the structure is built once and then queried many times. This project implements a leveled MPHF in Rust and compared it against two baselines: `boomphf-rust` and `bbhash-cpp`. The final evaluation uses a fair serial benchmark with separate runners for each implementation, along with a 16-seed benchmark to control for seed-dependent variance.

1 Introduction

A minimal perfect hash function takes a static set of keys and assigns each key a distinct integer in $[0, n)$. Unlike a standard hash table, an MPHF stores only enough information to reproduce that collision-free mapping for the original set. This makes MPHFs useful when the key set is fixed and the main goal is compact, fast lookup.

This project follows the leveled BBHash construction of Limasset et al. [1]. The construction works by hashing the remaining keys into a level, keeping the keys that land in singleton slots, and pushing collisions to later levels. This idea was implemented in Rust as `better-mphf` and compared it with two baselines: the Rust crate `boomphf-rust` and the original C++ implementation `bbhash-cpp`. The goal was not just to implement the algorithm, but to optimize it for practical static workloads in which construction occurs once and lookup dominates the overall cost.

The report does the following: first, it documents the implementation details, including the exact acceptance rule, retry-seed construction, and lookup-focused design. Second, it reports a fair serial benchmark setup with separate runners and a multi-seed benchmark. Third, it identifies which implementation choices helped (cached bucket IDs, in-place survivor partition, lookup bitset tuning, and `mul64` range reduction) and what still limits construction performance (seed-sensitive retry behavior).

2 Design and Implementation

2.1 Leveled BBHash Construction

The builder starts with all keys in a `remaining_keys` array. For one level, it hashes each remaining key into a slot array of size m. If a slot receives exactly one key, that key is placed in the level bitset. If a slot receives two or more keys, those keys survive to the next level. The process repeats until no keys remain.

Lookup replays the same sequence of per-level hashes. At each level, it tests whether the key's slot is set in the bitset. If so, a rank query converts that bit position into the key's final index by adding the cumulative number of keys placed in earlier levels. More levels mean more hashes and more rank probes per lookup.

2.2 Acceptance Rule and Retry Seeds

At a level with n remaining keys and m slots, the implementation counts the number of singleton slots and accepts the level only if the measured singleton count is at least the Poisson expectation

$$E[\text{singletons}] = ne^{-n/m}.$$

A candidate hash family is accepted only when it meets or exceeds that threshold. Relaxing the threshold reduces construction time, but it increases the number of levels, which hurts lookup. Because the target workload is static, slightly higher construction time in exchange for lower lookup cost is fine.

Good hash families should pass this test with high probability, so only a few retries are typically needed. If many retries are required, this usually indicates that the hash family or seeding strategy is inadequate. The retry cap (1024) acts mainly as a safety guard against such cases, not as a performance tuning knob. Testing cap values from 256 to 1024 had little effect on runtime, so the implementation keeps 1024 as a conservative default.

The current hash family is controlled by two constructor parameters, `seed` and `offset`. Internally they become a `base_seed` and a `retry_stream`, and the retry seed for level ℓ and attempt a is

$$\text{retry_seed}(a, \ell) = \text{mix}(\texttt{seed} \oplus \texttt{offset} \oplus \ell \oplus a\phi),$$

where ϕ is a 64-bit constant. Each accepted level stores its `retry_seed`, and lookup reuses that same seed.

2.3 Modifications for Speed

Several implementation choices differ from the original because the benchmark focus is the serial hot path.

Serial count/classify path. The serial builder uses a `u8` count array and a cached `seq_bucket_ids` array. During the count pass, it hashes every remaining key, increments a counter for that slot, and caches the slot id. If the attempt passes the acceptance rule, the classify pass reuses the cached slot ids instead of rehashing. It also compacts survivors in place with `seq_classify_pass_inplace`, writing collided keys back to the front of `remaining_keys` and then truncating the vector. That avoids allocating a fresh survivor vector on every accepted level.

Lookup-oriented bitset path. The bitsets are rounded up to one cache line (512 bits), the rank structure uses a fused `rank_if_set` query, and level 0 is peeled out in lookup because its cumulative rank is always zero. These are small changes individually but they improve the lookup path.

Fast range reduction. Slot mapping uses a multiply-high fast-range reduction rather than modulo. The implementation supports `low32`, `high32`, and full `mul64` variants; this follows the general idea of Lemire's nearly divisionless interval reduction [4]. The implementation defaults to `mul64`, as noted in later sections.

Auto-gamma schedule. Fixed gamma is still the fairest comparison because the baselines expose that interface. For `better-mphf`, the auto mode uses a piecewise schedule based on $r = n_{\text{remaining}}/n_{\text{total}}$. The schedule uses multipliers $\{2.0, 1.8, 1.55, 1.35\}$ for $r \in [0.75, 1], [0.5, 0.75), [0.25, 0.5), [0, 0.25)$ respectively. This gives the largest slot budget when the remaining set is biggest, which is the most expensive part of construction, and lowers the budget as fewer keys remain.

3 Benchmark Methodology

The benchmark harness uses separate executables for each implementation: one runner for `better-mphf`, one for `boomphf-rust`, and one C++ runner for `bbhash-cpp`. This avoids setup costs and makes fixed-gamma comparisons fair. The evaluation is only for the serial construction path and does not enable parallel features in any implementation.

The fixed-gamma benchmark is the primary comparison and baseline. Auto-gamma is reported separately for `better-mphf`. The 16-seed benchmark uses 13 input sizes from 512 to 2,097,152 and 9 gamma values from 1.15 to 2.50 on multiplicative keys. The evaluation also benchmarks seed robustness, key distribution, and fast-range mode. The claims in this report are based on multi-seed results because single-seed measurements are heavily distorted by seed luck.

4 Results

4.1 Fixed-Gamma Head-to-Head

The 16-seed benchmark contains 1872 total fixed-gamma cells (16 seeds × 13 sizes × 9 gamma values). Aggregated over all of them, `better-mphf` wins 1250 build cells while `boomphf-rust` wins 622. For lookup, `better-mphf` wins 1313 cells, `boomphf-rust` wins 515, `bbhash-cpp` wins 43, and 1 cell is an exact tie.

Those raw totals overstate the importance of small inputs where noise can affect numbers. A better view is to collapse each (n, γ) pair by taking the median across seeds. In the 117-cell median view, `better-mphf` wins 86 build cells versus 31 for `boomphf-rust`, and 86 lookup cells versus 29 for `boomphf-rust` and 2 for `bbhash-cpp`. At more practical sizes, the lead is clearer: for $n \geq 65{,}536$, `better-mphf` wins 45 / 54 build cells and 48 / 54 lookup cells.

This is the main result of the project. The implementation does not outperform the baselines in every corner case but the meaningful large-input trend is strong and lookup is the clearest advantage.

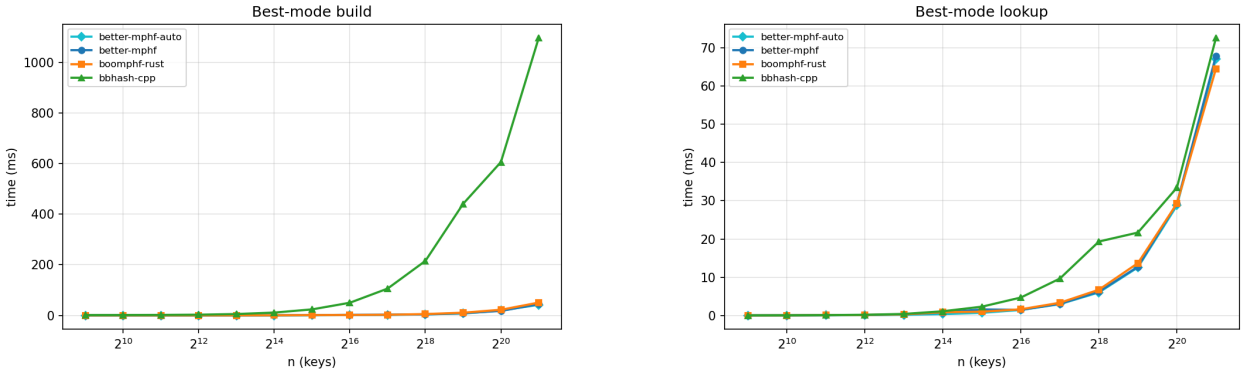

Figure 1: Left: best-mode build. Right: best-mode lookup.

4.2 Auto-Gamma Results

In the 16-seed benchmark, auto was compared against the best fixed gamma for the same seed and input size (208 comparison cells: 16 seeds × 13 sizes). Auto was better in 103 build cells and worse in 105, with a median build slowdown of 0.13% relative to best fixed. On lookup, auto was better in 156 cells and worse in 52, with a median advantage of 2.21%.

This makes auto a strong default when the goal is good lookup performance without hand-tuning γ. Auto is essentially tied with the best fixed gamma on build and is better on average for lookup.

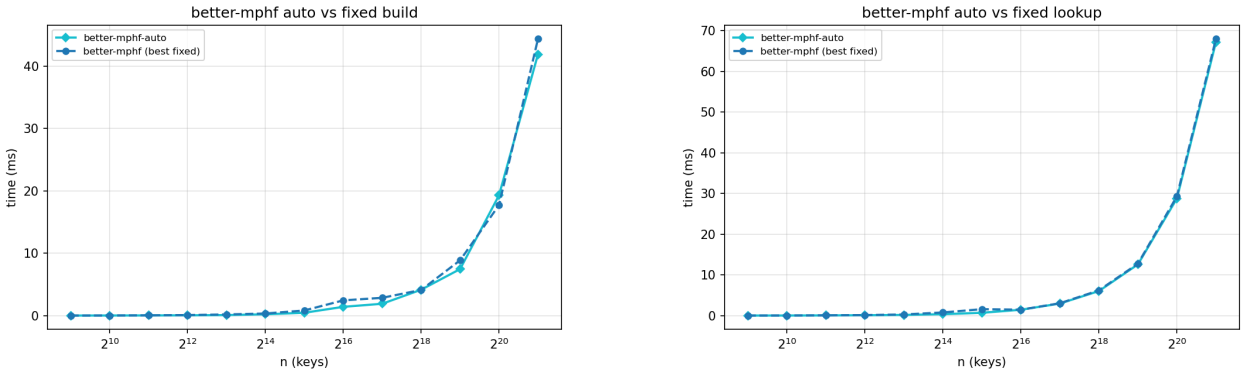

Figure 2: Left: `better-mphf-auto` build versus best fixed gamma. Right: `better-mphf-auto` lookup versus best fixed gamma.

4.3 Per-Gamma View and Workload Sensitivity

Figure 3 shows the fixed-gamma results in more detail. `better-mphf` performs best for medium to large n values, especially in lookup. The weakest cells are concentrated at very small sizes.

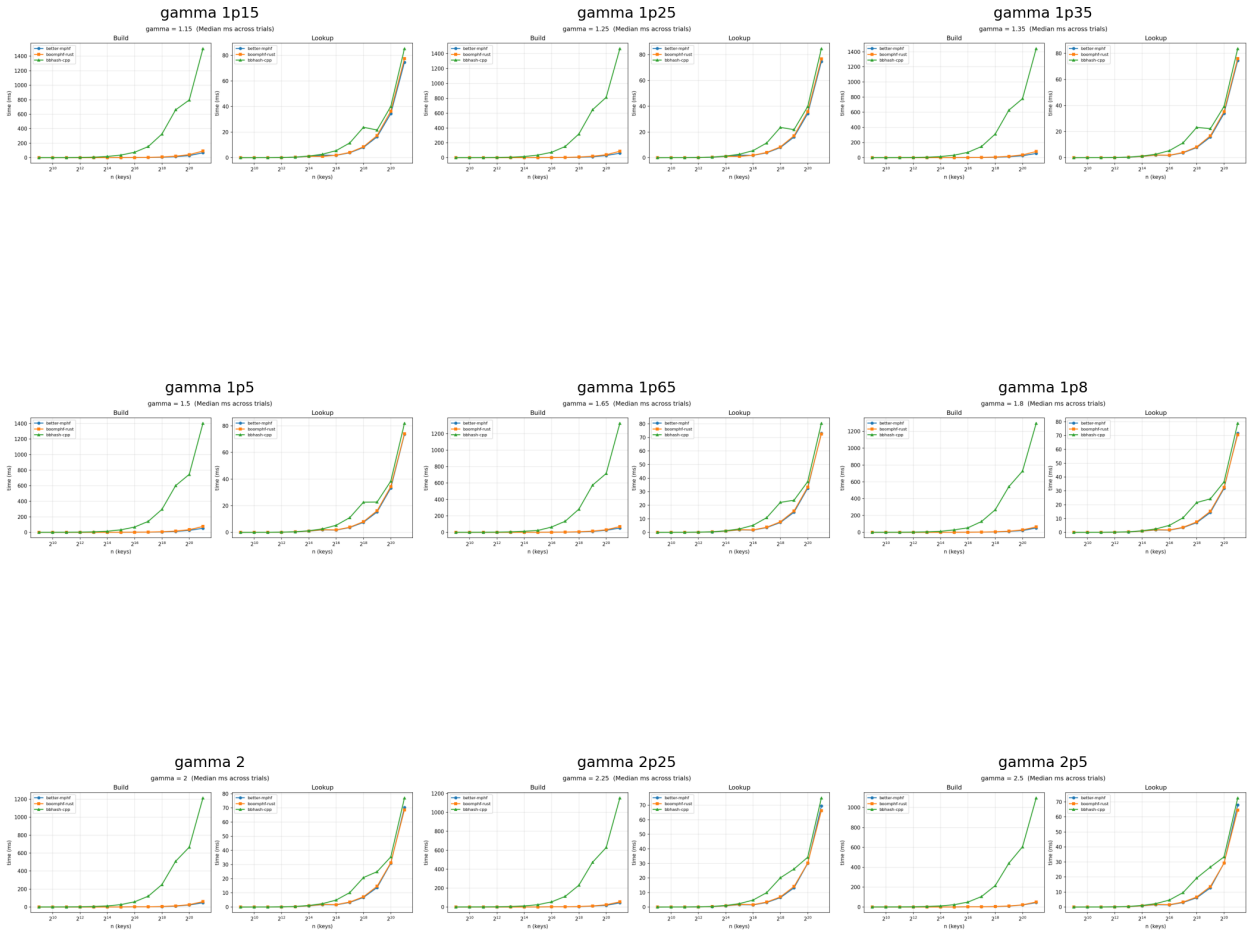

Figure 3: Per-gamma fairness plots from the 16-seed benchmark.

The fast-range comparison shows a clear pattern in the current data. In the 117-cell view, build winners split 54/44/19 among `mul64`/`low32`/`high32`, and lookup winners split 61/45/11 among `mul64`/`low32`/`high32`. Relative to `low32`, `mul64` improves median build time by 7.4% and median lookup time by 0.2%, while `high32` improves build by 5.9% but hurts lookup by 1.8%. In this comparison, `mul64` is the strongest overall mode.

The key-distribution comparison tells a similar story. Lookup is very stable across tested modes: relative to multiplicative keys, the median lookup change stayed within about $\pm 0.2\%$ for sequential, clustered, high-bit-heavy, and splitmix-random keys. Build is more workload-sensitive, and every tested alternative distribution was slower than multiplicative keys, with median slowdowns from $+13.0\%$ to $+32.0\%$. No distribution produced a failure mode.

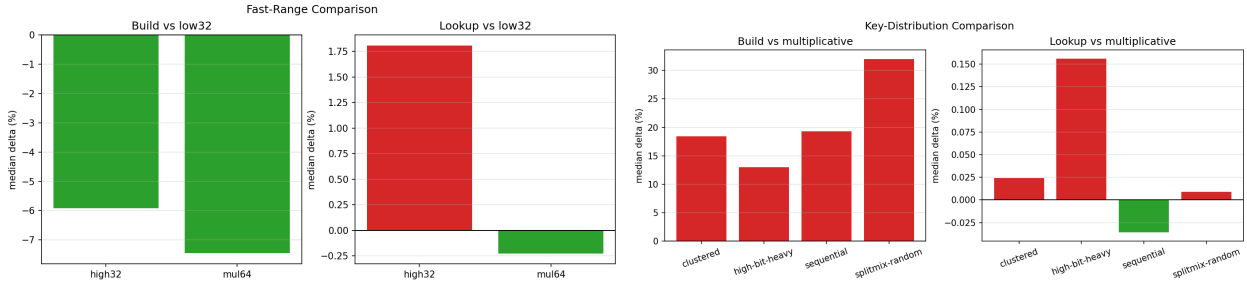

Figure 4: Left: fast-range comparison. Right: key-distribution comparison.

4.4 Seed Robustness

Seed sensitivity is the main weakness of the current builder. In the 16-seed benchmark, the worst/best build-time ratio across seeds has a median of 2.98× and a maximum of 11.01×. At $n = 1{,}048{,}576$, the fixed-gamma benchmark gives representative per-gamma ranges: at $\gamma = 1.35$, best/median/worst across 16 seeds is 22.1/25.6/33.5ms; at $\gamma = 2.00$, it is 17.4/22.8/50.9ms.

This spread shows why one fixed seed is not enough for evaluation. A single unlucky seed can make a stable implementation look much worse than it really is, which is why the paper relies on multi-seed results.

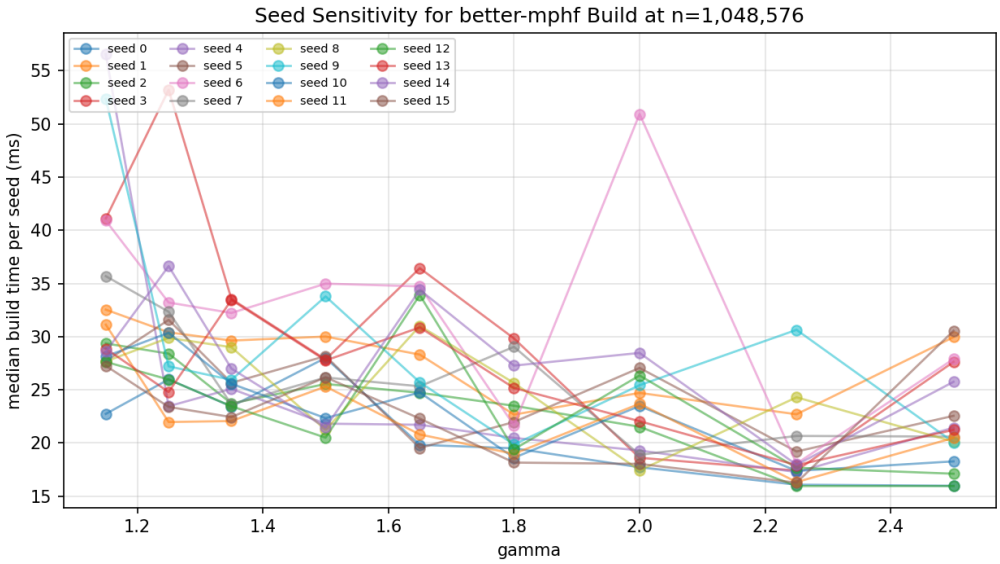

Figure 5: Seed sensitivity for `better-mphf` build at $n = 1{,}048{,}576$.

5 Conclusion

The most important outcome is that the Rust implementation now meets the intended static-workload profile reasonably well. Lookup is the strongest phase: in the seed-median view, `better-mphf` wins 86 of 117 lookup cells overall, and 48 of 54 cells at $n \geq 65{,}536$, where input sizes are large enough for timer noise not to dominate. Construction is competitive for medium and large inputs: 86 of 117 build cells overall, and 45 of 54 at $n \geq 65{,}536$. At the small-n values, there is a lot more noise and the wins are less meaningful.

The remaining problem is that unlucky seeds can increase construction time. The next focus should be reducing seed-dependent variance while keeping strict Poisson acceptance unchanged so that level count and lookup quality do not regress.

References

[1] Antoine Limasset, Guillaume Rizk, Rayan Chikhi, and Pierre Peterlongo, *Fast and Scalable Minimal Perfect Hashing for Massive Key Sets*, 16th International Symposium on Experimental Algorithms (SEA), 2017.

[2] `boomphf` Rust crate, `https://crates.io/crates/boomphf`.

[3] BBHash C++ library, `https://github.com/rizkg/BBHash`.

[4] Daniel Lemire, *Fast Random Integer Generation in an Interval*, ACM Transactions on Modeling and Computer Simulation, 29(1), 2019.